University of Pittsburgh
ECE1895: Junior Design Fundamentals
Mari Inglese, Yasseen Sabil, Nicholas Starace

Design Project 2: Bop It

I. Overview

I.1 – Original Design

The design and product presented here for project two details Cook-It, a game where players are prompted with audible and visual cues to strike or tilt various areas on a cutting board. The game ends when a player reaches a score of ninety-nine points or three incorrect gestures have been performed. This game was inspired by the popular toy Bop It and video game Overcooked, where players work collaboratively to prepare restaurant dishes under a tight time constraint.

Audible instructions that act as a head chef call out one of four unique verbs to provide a player with information on the required actions. Three of the four callouts are “Chop-it”, “Cook-it”, and “Crush-it”, corresponding to cutting zones one, two and three, respectively. Once the player knows the assigned cutting zone, they can chop, cook, or crush potential items in the queue, indicated by LED strips in each zone. The fourth and final callout is “Clear the plate”, giving players a chance to tilt the cutting board and clear the queue of orders, turning all LEDs off. Early visualizations of the game are shown below in figure 1. Initial designs also included an additional LED strip located below the striking zone where a player could see accumulated LEDs for each successful action.

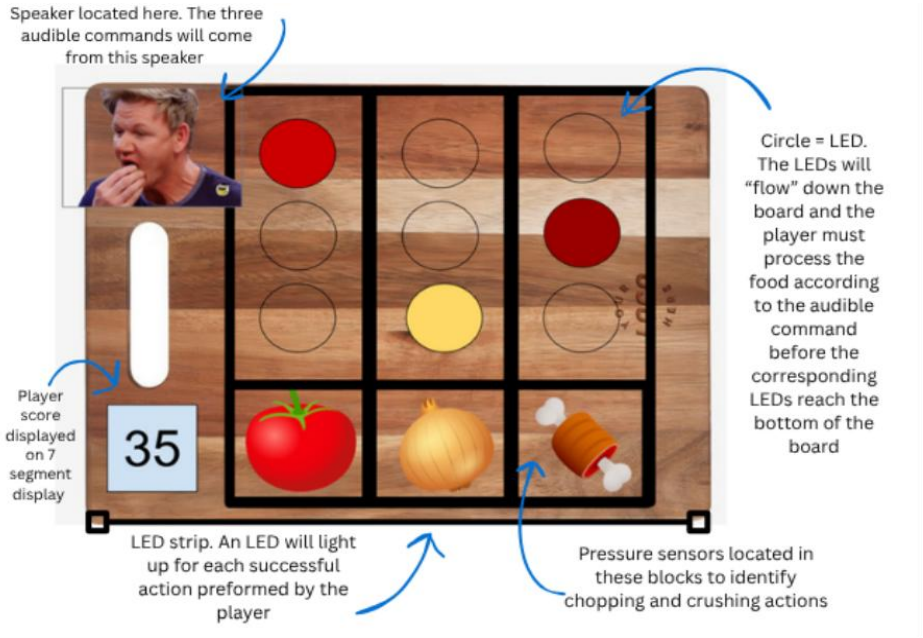

Figure 1: Initial prototype for Cook-it layout.

II. Design Verification

II.1 – Development of LED Strip Queue

In the initial prototype, each sensor input, score display, audio circuit, and LED strip was developed independently before integration. Given the number and control complexity of LEDs required to implement the color queue, an addressable LED strip programmed with I2C enabled the team to control the color and brightness of 60 LEDs while only occupying two pins on the microcontroller.

The LED strip used here was the APA102, giving the team access to various open-source examples. Additionally, the APA102 library simplified the software required to program custom animations into the strip since all lights are represented by a single array. For initial verification, a single zone of ten LEDs was programmed to populate at the top index, fall to the bottom, and remain at the bottom until the zone was full. A game over sequence was also prototyped to have all lights flash red. The expectation to expand functionality to three zones was by copying the structure of code for one zone while randomizing the sequence of LED addition to ensure that no columns were identical.

II.2 – Development of Sensor Input

The team prototyped a RP-S40-ST pressure sensor, BMI270 IMU, and US1881 hall effect sensor, each in isolation, to confirm the hardware setup and develop the software associated with each of the sensors. The pressure and hall effect sensor outputs were connected to open GPIO pins on the ESP32-S3 development board. The IMU, communicating with the ESP32 over I2C, was connected to dedicated SDA and SCL pins. Functions were written to process the data output by each sensor and to integrate the sensors with the LEDs and the score display.

Through prototyping, the team realized a software debounce was needed to ensure the sensors functioned as intended. A debounce delay of fifty milliseconds was added after each change in voltage from the pressure and hall effect sensors so that a single action performed by the user would not cause the score to unduly increment.

The prototype was also used to calibrate the IMU. Sensor data from the IMU was collected and printed to the serial monitor to determine a threshold for tilt detection. Rotation about the y-axis was determined to be the most useful metric for detecting if the user tilted the board because it had the largest change when the tilting motion was performed. However, the tilt detection threshold that was set during the prototyping phase didn’t consider how the IMU would be mounted within the enclosure. Once the IMU was secured to the enclosure, the amount of rotation it experienced when the full board was tilted was not enough to trigger the tilt detection and the threshold needed to be adjusted.

II.3 – Development of Score Display

Due to component availability, the two digit score display was prototyped using two common anode seven-segment displays and controlled by the SN7446 driver. To display a digit on the seven-segment, four bits were required to represent zero through nine. Ultimately, the incrementScore function was developed to convert the integer score to an array of eight bits and write the output to the driver. The code for this function is shown below in figure 2. The function loops through each index of the bit array by shifting the integer to the bits' respective position and writing it to the driver.

```
void incrementScore() {
  currScore++;
  for (int i = 0; i < 8; i++) {
    bool bit = (currScore >> i) & 1;
    digitalWrite(ScoreDisplay[i], bit ? HIGH : LOW);
  }
}
```

Figure 2: Integer to Binary Score Conversion

II.4 – Development of Audio Circuit

The DFPlayer Mini module was selected to handle all audio for its ease of use and integration with the ESP32 and other microcontrollers. Initially, the team could not establish communication between the DFPlayer and ESP32 after trying to define a simple serial instance. Eventually, the team discovered that the DFPlayer required a specific HardwareSerial class to initialize communication with UART and successfully established communication with the DFPlayer. Various audio commands were recorded on the computer by members of the team and uploaded to the microSD card.

II.5 – Microcontroller Selection

With all components working individually, the team decided that an ESP32-S3-WROOM-N8R8 was the best microcontroller for this application because it provided enough I/O pins and could be programmed in the Arduino IDE. Other reasons for using the ESP32 included its capacity for digital audio communication (I2S) that were not available on microcontrollers such as the Atmega328P.

III. Electronic Design Implementation

III.1 – Initial Design Conversion from Breadboard to PCB Schematic

The goal of this PCB design was to achieve functionality identical to the prototype while making all circuit elements as integrated and compact as possible. The first stage of PCB design consisted of copying the prototyped design into the Altium schematic while adapting components for PCB integration. Shown in figure 3 is the first and main sheet of the PCB schematic, detailing all controllers used in the design.

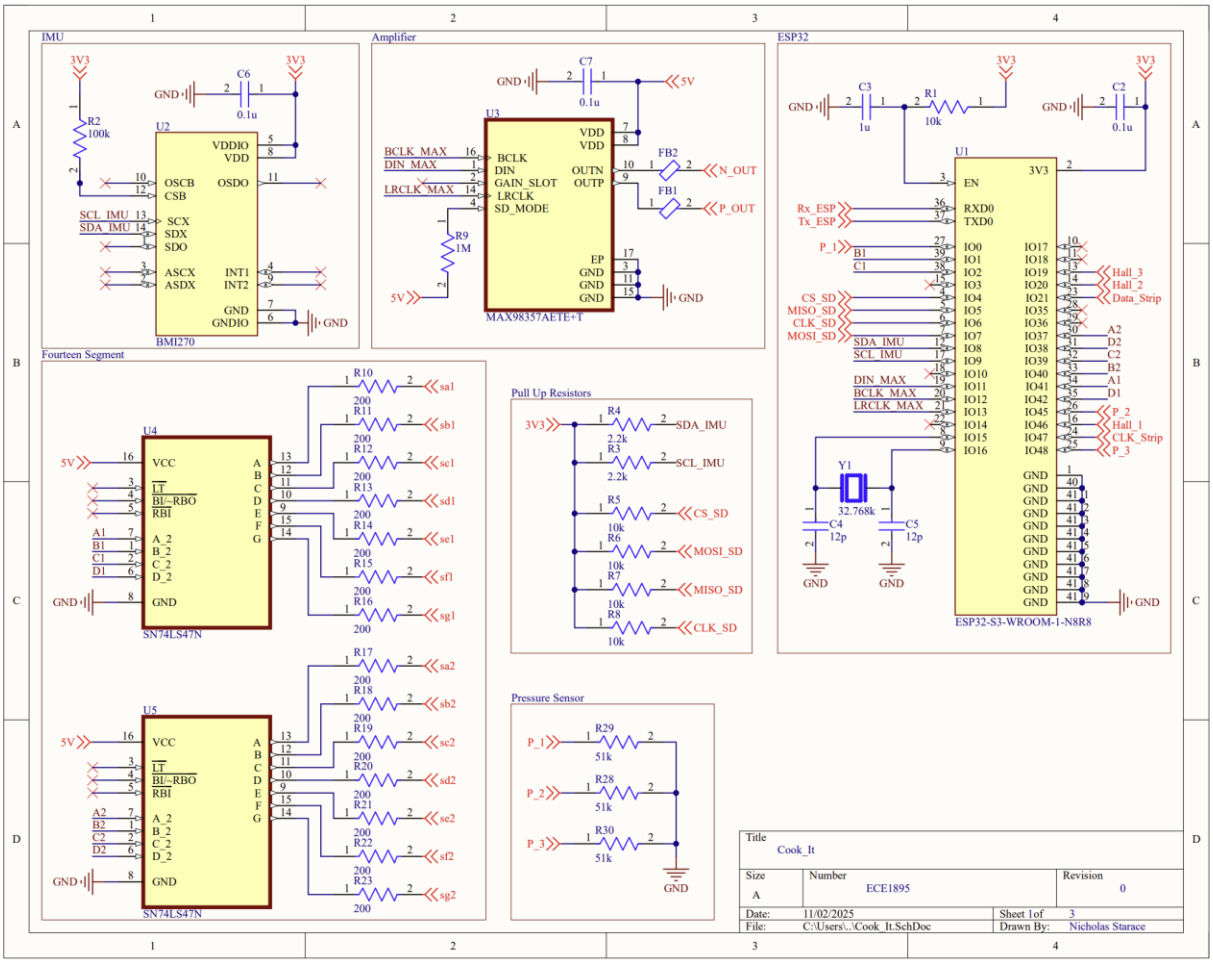

Figure 3: PCB Controller Schematic

The first PCB adaptation was the refinement of the full ESP32 development board used in the prototype down to an isolated ESP32 microcontroller. For IMU integration, the application schematic from the main controller datasheet was copied into this schematic. The seven segment score drivers were changed to the surface mount package and imported into the design. Lastly, an external clock was added to the ESP32 for increased timing precision, if necessary.

III.2 – DFPlayer Alternative

Since the team was unable to have the DFPlayer circuit functional before the PCB was ordered, an alternative audio circuit design was implemented. The new amplifier design consisted of reading directly from an SD card over SPI to the ESP32, converting the data to I2S (a popular audio protocol that only requires three wires), and passing it to the MAX98357 to output desired sound. The MAX98357 seemed to be a good choice due to its ability to both decode and amplify the data signal. With this approach, no data would have to be buffered in the ESP32, and the only required software would be to convert SPI to I2S.

III.3 – PCB Connectors

Shown in figure 4 is the second sheet of the PCB schematic detailing external connections. Since the pressure sensors, led strips, buttons, battery, and hall effect sensors occupied a significant area of the enclosure, it was impossible to embed these components into the PCB. Therefore, curved TSW headers were used for all external connections. TSW headers were chosen due to their ease of use, providing a simple and reliable access point for jumpers. The TSW was chosen over the popular JST connector since it is faster to setup, requires less wiring, and provides access to individual pins. Additionally, using a curved TSW header meant that the PCB would have a shorter height and could be fit into a smaller area.

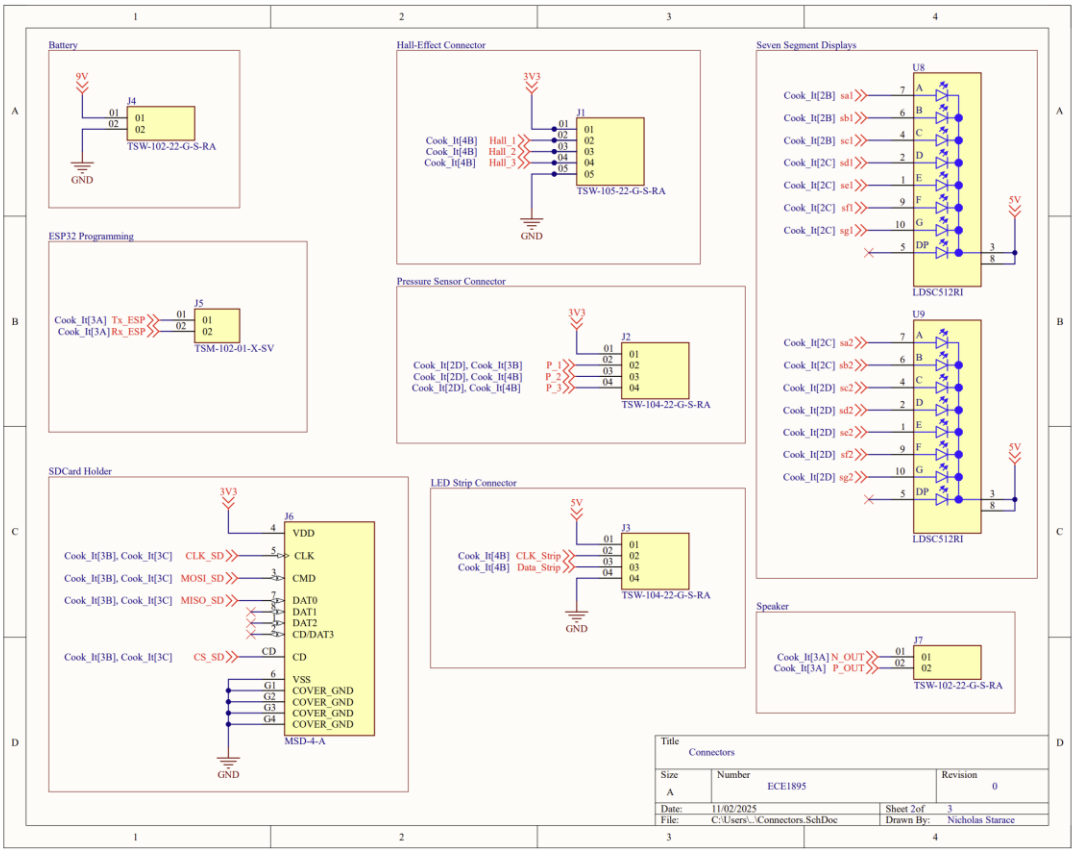

Figure 4: PCB Connector Schematic

III.4 – Voltage Regulation

All components connected to this PCB require either 5 V or 3.3 V. The input to this PCB was intended to be a 9 V battery, therefore requiring two regulators to step the supply down to 5 V and 3.3 V. The third and final sheet of this PCB schematic is shown below in figure 5 detailing the voltage regulation present on this board.

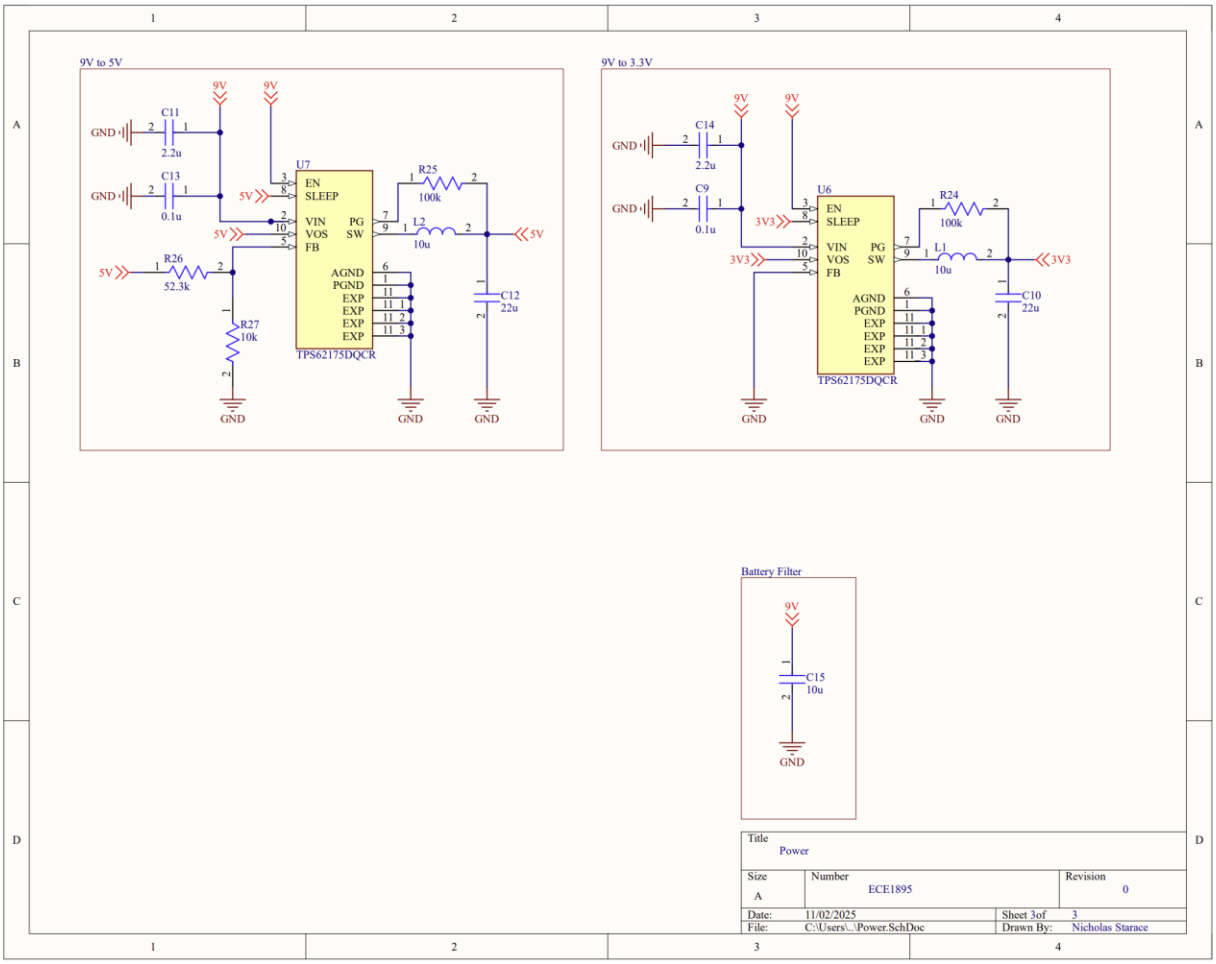

Figure 5: PCB Power Schematic

The TPS62175 was selected for its ability to be configured for an output voltage between 1 V and 6 V. The step-down converter had the ability to accommodate any voltage input between 4.75 V and 28 V, placing the 9 V battery in an acceptable input range. Additionally, the regulator was capable of outputting up to 0.5 A of current, sufficient for our power consumption.

According to the regulator's datasheet, the default output voltage is 3.3 V, making the design for the respective supply rail simpler than the 5 V rail. To configure the regulator to output 5 V, a voltage divider was designed for the reference pin according to the datasheets application guide and governing equation shown in Eq. 1.

$$R_1 = R_2 \left(\frac{V_{out}}{V_{ref}} - 1 \right) (Eq. 1)$$

The default output voltage of the regulator (V_{ref}) was set to 3.3 V, the default output voltage of the regulator. The desired output (V_{out}) was set to 5 V. R_2 was fixed at 10 kdata sheetested by the datasheet to improve noise immunity. Solving the equation yielded an R_1 value of 52.3 kΩ. All other components connected to the regulator shown in figure 5 were as suggested by the datasheet.

III.5 – PCB Layout

As mentioned earlier, one of the goals of this PCB design was to minimize size. Shown below in figure 6 is the PCB trace layout. The design approach of this PCB layout prioritized connectors to be on the border of the board and proceeded with modular integration. Modular integration is defined by laying out components local to their respective circuit. For example, the 5 V regulator and all its components defined by the bounding box in the schematic were placed according to the suggested datasheet application. Once a controller's filtering capacitors, supplying inductors, and resistors were laid out, the package could be moved around to fit the overall chips layout.

Three different trace widths were used in this design with the largest having a width of 1 mm to lower the impedance of the supply rails. The smallest had a width of 0.25mm to accommodate the small size of the amplifier and IMU controllers. 0.1 uF filtering capacitors were placed on the supply pin of every controller to stabilize the voltage supply, placed close to the pin. Finally, controllers were placed near the pins of the controller they were communicating with, naturally leaving the ESP32 to be located in the center of the board since all communication went through it.

III.6 – PCB Design Verification

While all figures presented in this section show the fabricated version of the PCB, various pins on the ESP32 were changed from their original assignments on the prototype for the purpose of trace routing. By changing the pin assignments and confirming that the new I/O was suitable for the signals communication protocol, trace length, complexity, and size of the board could all be reduced.

In running the design rule check for fabrication, the pads of the regulators, amplifier, and IMU were all too close to one another and failed the rule check. Rather than reselecting different components and redoing the PCB layout which would have added hours to the design process that the team did not have, the ground pad in the center of the chips were made smaller to pass the rule. This seemed to be a safe decision since the pad was significantly larger than the other signal pads and the remaining pad surface area was adequate.

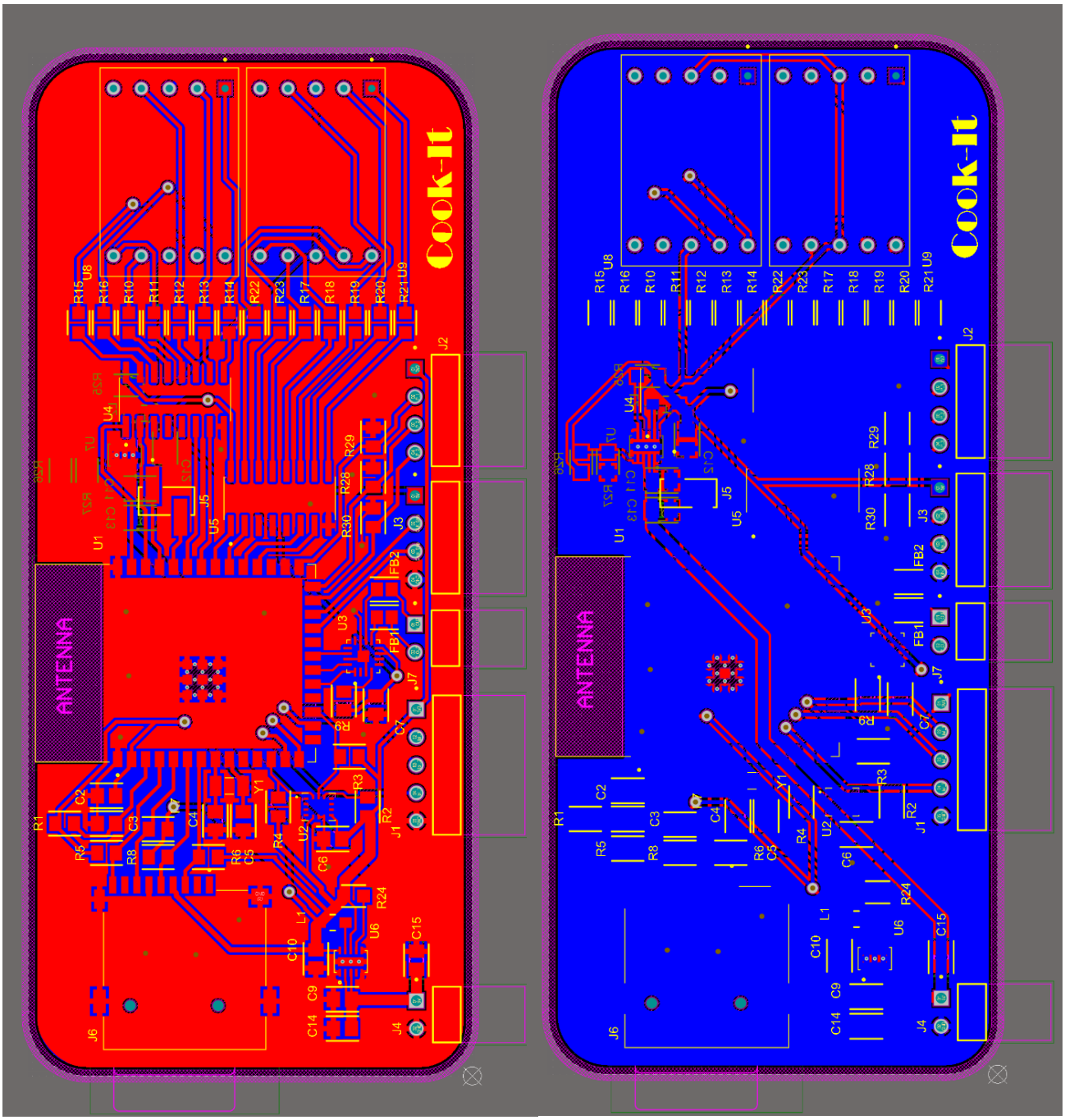

Figure 6: PCB Trace Layout

IV. Software Implementation

The overall structure of the game was based around three "hit columns," referring to the LED-signified queues and corresponding sensor input zones on the cutting board. Due to the nature of the Arduino coding environment, the main loop of the program had to continuously check for sensor inputs and alter global variables to track the status of these inputs over time. If a sensor was activated (outside of the debounce range), the current time was recorded. Then, the time since the last strike was calculated and a logical operation was carried out.

Pseudocode:

```
IF time_since_last_strike > minimum_time AND time_since_last_strike <
maximum_time THEN
    hits[column] = hits[column] + 1
ELSE
    hits[column] = 1

IF hits[column] >= 3 THEN
    incrementScore()
    hits[column] = 0
```

By defining the variables of the hit window specifically, the rate at which the rhythmic hits must be carried out can be optimized for feasibility and enjoyability, and this was later verified via physical gameplay testing.

The fundamental variable within the gameplay loop was a Boolean array that represented the three hit zone columns indicated by LEDs on the board. This data, alongside the strike counter and the score, fully defines the game state of the simulation. Simple functions were designed to manipulate the Boolean array according to the actions that took place in the gameplay loop. One function, `addItemsToColumn`, was designed to run after some interval of time, which decreased as the score increased. This was implemented by continually performing a check in the loop for whether the current time was greater than the previous item-add time plus a defined interval. If so, the last item-add time was reset and the action was carried out. This type of functionality was used for all periodic actions in the program to avoid using the `delay` function, as this prohibits any sensor input during the duration of the delay.

To validate the creation of the gameplay loop without the full prototype of the PCB, a digital version of the game was created, taking manual inputs from the keyboard of a computer. This proof of concept allowed for verification of the hit-processing algorithm, the plate-cleaning minigame, and the queue visualization functions prior to the full working prototype.

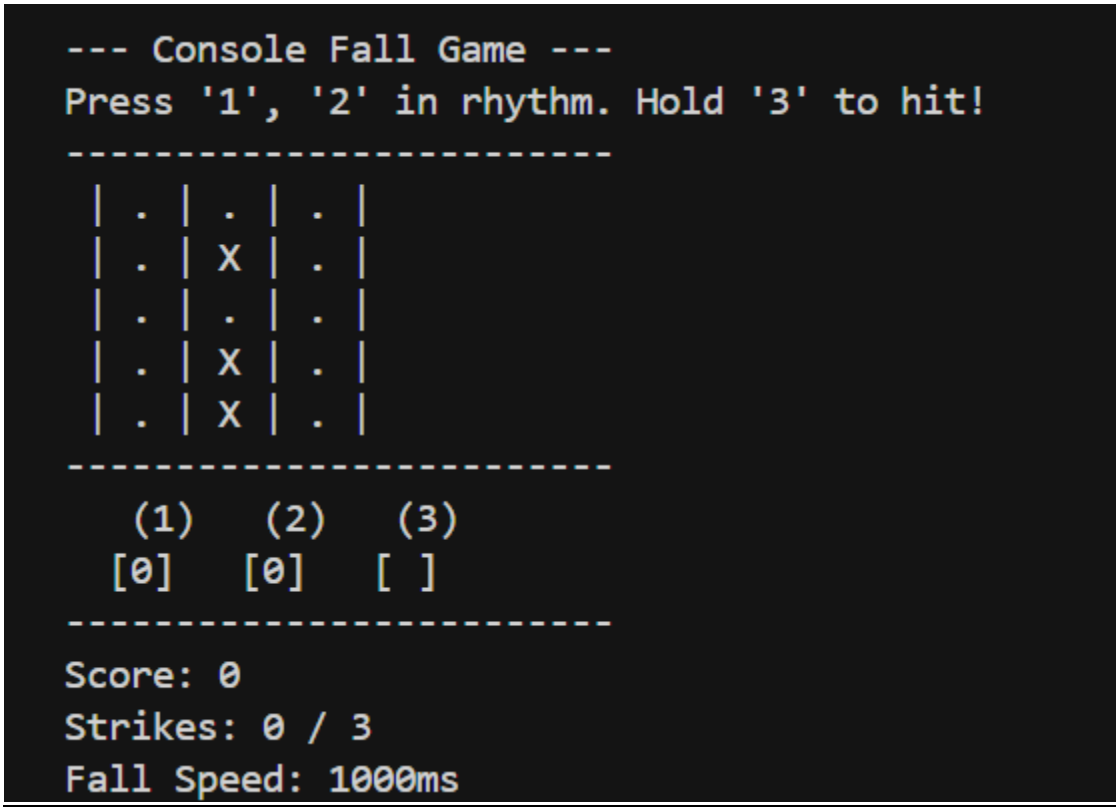

Figure 7: Terminal version of the Cook-It game using keyboard inputs.

To transform the digital version into the PCB version, the section of the code which took manual inputs from the keyboard was rewritten to take inputs from the pressure sensors, button, IMU, and hall effect sensor.

The outputs to the terminal were replaced with one game visualization function, which occurs once every loop. This function sent a signal to the LED queue, which set three segments of the LED display to be used for the columns. This was written to be easily manipulable by a single vector of starting indexes, so the exact placement of the LED queues could be shifted once the board was constructed.

Furthermore, this implementation also required that the middle strike column queue segment operate in reverse, as the LED strip was physically placed in a snake formation inside the enclosure, making the orientation of the column strip different from the outer columns.

The amount of strike penalties the user has accrued is also visualized by enabling a red LED underneath the ‘X’-shaped holes in the enclosure on the left-hand side. The software for this was created in a manner similar to the rest of the LED strip functionality. A starting index was set and incremented to match the actual placement of the strip.

Furthermore, the gameplay design included a feature which defined the ‘Active Zone’ at any point in the game. This is the area of the board which can be struck at any given moment. This changes at a regular interval of five seconds and hitting the board outside of this region will add a strike penalty to the user.

The addition of new items and the rate of items dropping down the column both increase as the game continues, and players will face a penalty if any column is full while an item is added. Therefore, the player is required to hit the active zone faster, risking a penalty if the active zone switches during a tap motion or if they select the wrong zone to hit.

The “Clean the Plate” minigame was implemented via the ‘Active Zone’ command. One-sixteenth of the time, a command will be called out to ‘Clean the Plate,’ which was achieved using random variables drawn at defined time increments.

Once this command was called, the program would enter a while loop which would only be exited when one of the following conditions was met:

- The IMU gave a signal indicating that a board tilt has occurred (Plate was cleared).
- A designated interval of time had passed since the loop was entered (Plate was not cleared).

V. Enclosure Design

V.1 – Enclosure Design Version 1

The initial design of the enclosure was based on our early prototype design shown in Figure 1. The CAD file for the bottom of the enclosure was created in Easel and is shown in figure 8.

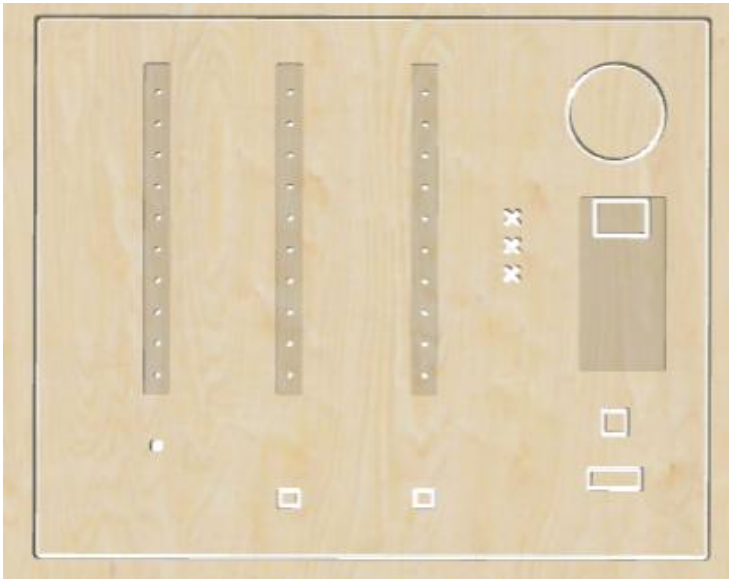

Figure 8: Enclosure Revision 1

The bottom plane of the enclosure is where the LED strips, speaker, PCB, score display, start button, on/off switch, pressure sensors, and hall effect sensor are mounted. The LED strips, and PCB were mounted in the enclosure in recesses that were cut to half the thickness of the board. The LED strips lay face-down in the long column recesses so the light from the LEDs is visible through the holes drilled in the columns. The PCB lays face-down in the rectangular recess. The score display is soldered to the front of the PCB, so the score display is visible through the rectangular cutout in the plywood.

There are rectangular cutouts for the hall effect sensor, the start button, the on/off button, and the speaker. The pressure sensors were mounted on the top of the enclosure and the leads connected to the pressure sensors were inserted into small rectangular cutouts. Two rectangular cutouts aligned at 45-degree angles were added to create a strike counter next to the score display.

The first version of the enclosure didn't include the correct tolerances on the cutouts, so the LEDs and buttons didn't fit into their cutouts. Additionally, the origin was misaligned during manufacturing so one of the sides of the board was cut too close. The cutout for the on/off button was damaged as a result and there was no margin available to mount the sides of the board.

V.2 – Enclosure Design Version 2

The second revision of the cutting board enclosure resized various cutouts for the LED strip, PCB, and speaker based on how the components fit in the first revision. Additionally, the walls and bottom panel of the cutting board were designed and manufactured using the same process as the first revision. A rendering in easel of the second revision is detailed below in figure 9.

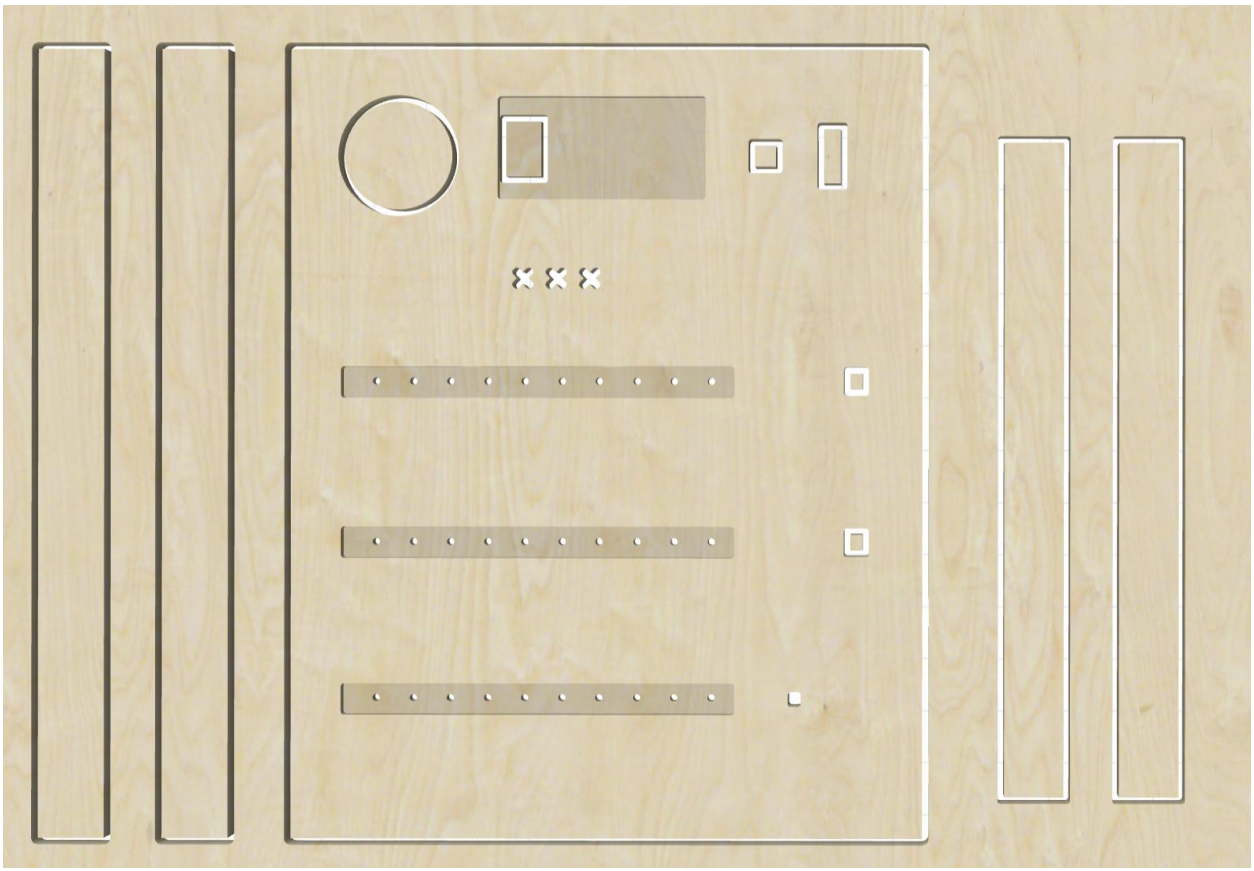

Figure 9: Enclosure Revision 2

V.3 – Enclosure Materials and Manufacturing

The team settled on birch plywood as the enclosure material due to its similarity to a real cutting board. To make the material even more similar, the team used wood stain to darken the color of the wood. While the plywood could have been drilled using a CNC machine or laser cut, the team opted to use a X-Carve CNC machine since the detailed design was easier to execute in the Easel software compared to its laser cut alternative.

VI. Assembly and System Integration

VI.1 – Enclosure Assembly

After two enclosure revisions, the cutting board box was assembled using wood glue to bond the four pieces together. Prior to bonding the pieces, the location of each joint was changed to only expose the top lid of the cutting board. Therefore, the sizes of each element no longer fit one another. To resize the pieces, hand sanding would have been a long process and terrible uniformity across. A faster and more precise alternative was to use an air sander that gave better control, speed, and power over the sanding process.

Once the pieces were resized, the box was bonded together using wood glue and held in position with clamps perpendicular to the joint. To provide repeated access to the inside of the board, Velcro was attached at each corner of the box using superglue. Finally, the box was stained using for an aesthetic appearance.

VI.2 – PCB Assembly

Assembling and programming the PCB took multiple attempts and culminated to a failed integration with the overall game. The two tools used to assemble the PCB were a stencil, soldering iron, heat gun, and reflow oven. The PCB was first surrounded with additional copies of the same board to provide a flat surface for the stencil. These peripheral boards were secured to the table and stencil secured to the peripheral boards using masking tape. A small amount of solder paste was smoothed across the stencil using a squeegee. After applying solder paste, the stencil was removed and components placed using tweezers. Finally, the board was reflowed in the oven and through hole components soldered as shown in figure 10.

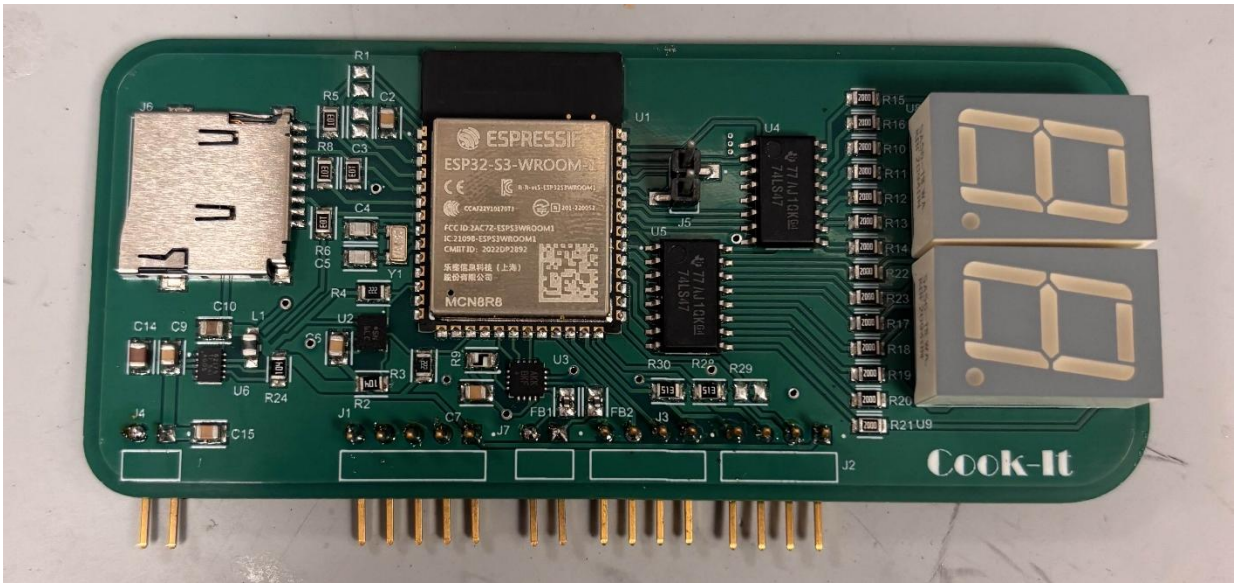

Figure 10: Assembled PCB Front Side

With components attached to the front side, it was impossible to lay the PCB flat and repeat the same assembly process for the back side. Given the small number of components on the back side and their dense concentration, the stencil was cut small enough to be secured to the board itself. The back side assembly is shown in figure 11. Solder paste was applied in the same manner as the top side assembly while the board was held stationary with a helping hand. After the stencil was lifted and components placed on their pads, the team opted to use a heat gun to bond the components since reflowing could risk heavy components falling from the underneath.

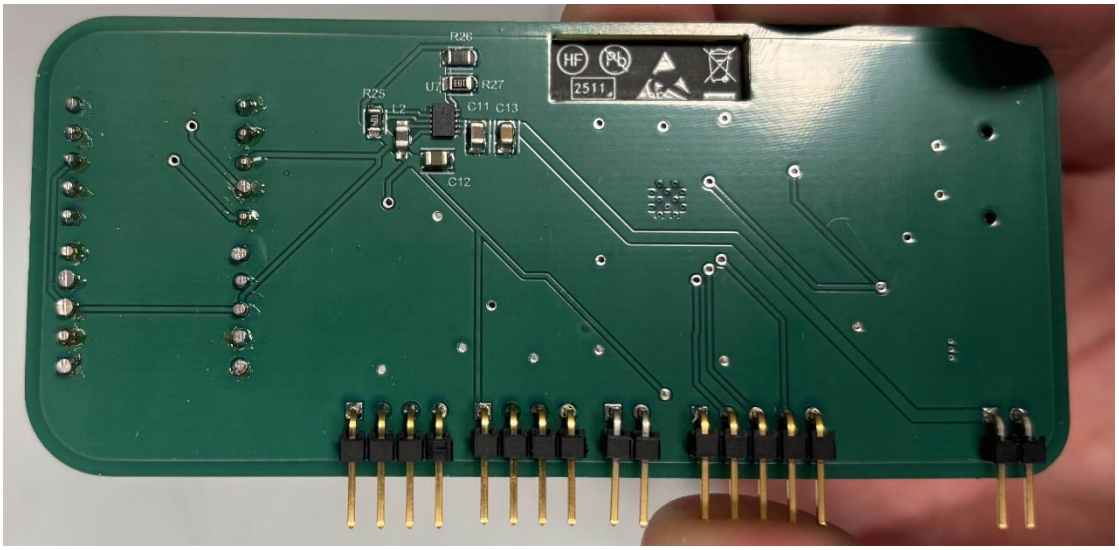

Figure 11: Assembled PCB Back Side

VI.3 – PCB Troubleshooting and Programming

Upon supplying the PCB with 9V power, the 5 V regulator output the correct voltage while the 3.3 V regulator output 5 V and pulled 150 mA. Looking back to the datasheet, the application used for this regulator was for the TPS62177 (not the TPS62175). Since the pads of the regulator were too small to solder a wire and all electronics on the 3.3 V rail were likely burned, the team ordered a TPS62177 along with new 3.3 V components (ESP32, IMU). After reflowing the updated PCB, the TPS62177 output 2.4 V, indicating a current leak. The input filtering capacitor (C9) was removed to combat the issue since it was the only component not specified in the application guide and had a capacitance smaller than the suggested value. Finally, the output voltage was a non-constant 3 V ranging between 3.14 V and 3.24 V.

To program the ESP32, the chip had to be placed in boot mode requiring access to the enable (EN) and boot (GPIO00) pins. A wire was soldered to the enable pad, jumper connected to the boot pin and wired to a programming circuit as shown in figure 12. The PCB design planned to program the chip using the ESP32 development module's UART chip to convert USB to serial. To bypass the ESP32 on the development module, the enable pin was pulled low while Tx and Rx connected to the pins on the PCB ESP32. Figure 13 shows the full setup of the programming circuit and development module circuit connected to the PCB with the red button connected to enable and black button connected to boot.

Due to the unstable 3.3 V regulator output, it was not possible to program the ESP32 using the power electronics on the PCB. Therefore, the regulator was bypassed with a power supply connection soldered directly to the 3.3 V rail. At this stage, the ESP32 communicated occasionally with the laptop and boot messages were displayed in the serial monitor. The sporadic communication with the ESP32 and time constraint led the team to implement the breadboard prototype in the final product, ensuring that a working game was submitted.

VI.4 – Enclosure Integration

The breadboard with all electronics was secured to the base of the enclosure using Gorilla tape along with the LED strip. Other components such as the speaker and buttons that had a smaller surface area were secured with a combination of hot glue and superglue. Finally, images for strike zone locations were printed and bonded to the wood using wood glue.

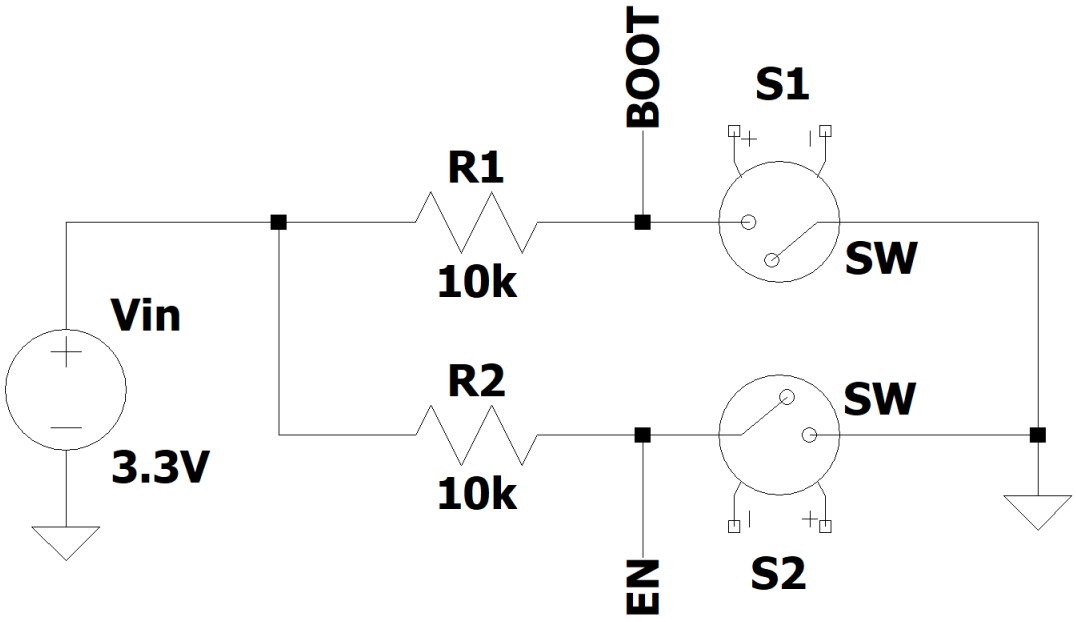

Figure 12: ESP32 Bootloader Circuit

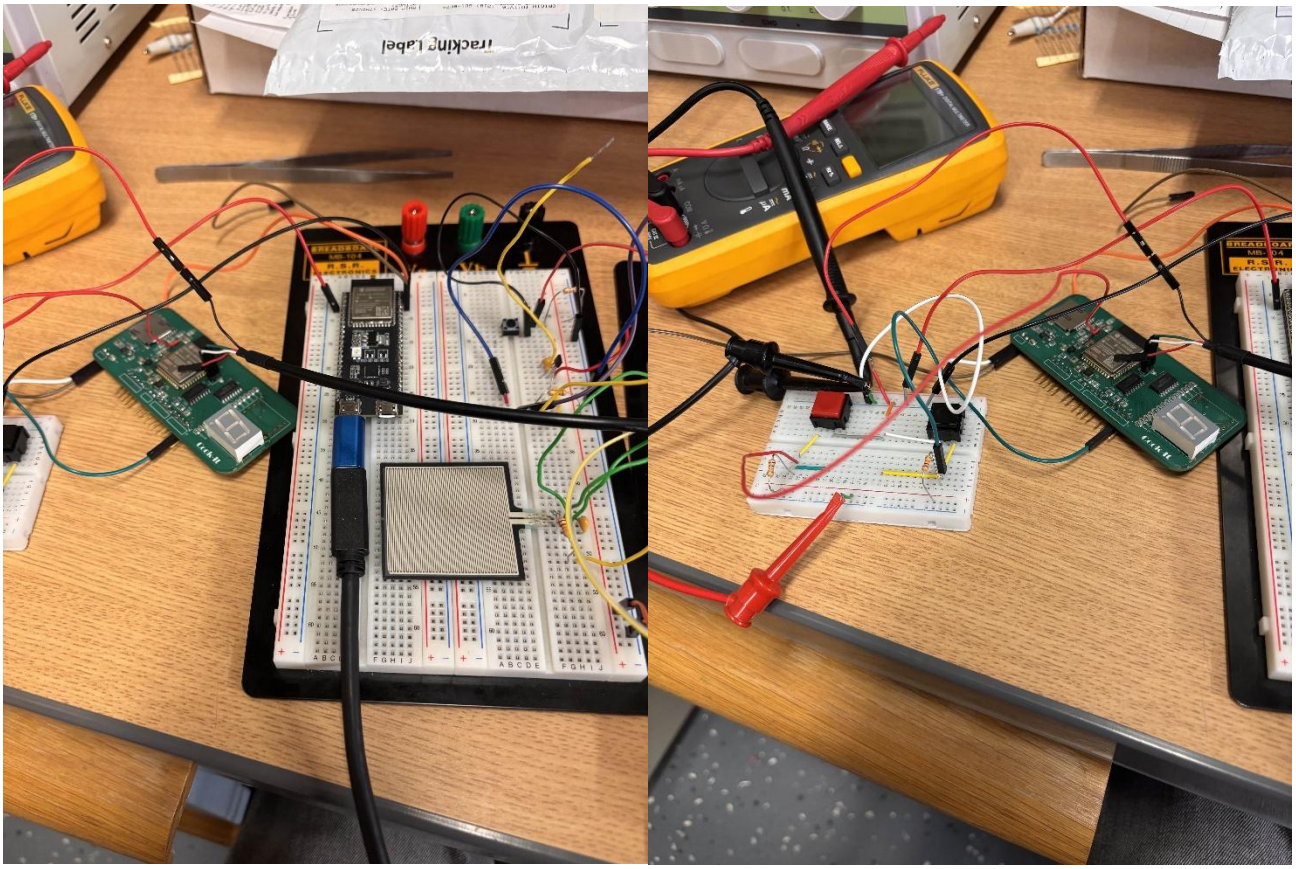

Figure 13: PCB Programming Setup

VII. Design Testing

The testing and verification process for the Cook-It device was two-fold. Firstly, the individual sensors and components had to be configured, and their functionality had to be verified independently. The software which combined these components was also created and tested independently with manual controls through a keyboard and outputs displayed visually in a console terminal, as described in the software section of this document. Secondly, the components had to be tested and verified in an integrated manner.

The first stage of the testing occurred once the materials ordered arrived. The new components were distributed amongst the team members. This included the IMU, the pressure sensor, and the LED strip. Each team member was tasked with reading input/sending outputs to their device with an Aptmega328 IC chip and manipulating this value via software. For the pressure sensor, the configuration required the precise selection of a resistor to place in series with the pressure sensor. This resistor would have to be of value which was in between the high and low value of the pressure sensor's resistance to change the binary reading of the output signal would change when pressure was applied on the resistor.

With the IMU and LED strip, libraries were found which were used to read/send signals to/from the device. Once these packages were found, programs were written to verify the basic behavior and show that they could be manipulated within the Arduino framework. The result of this initial testing allowed the group to continue to hardware integration.

Secondly, the integration process began. As components were added together, the digital game version was modified to include physical inputs instead of keyboard inputs and was loaded onto the PCB in an iterative manner. Upon the introduction of each component/change to the gameplay loop code, the program was ran again and the previously established functionality was verified to still work in conjunction with the new functionality. This testing was carried out before the final assembly within the enclosure, due to the difficulty in modifying the circuit once it has been placed in the enclosure. This testing may have led to some diminished capacity in the final version of the project such as having to replace the hall effect sensor with a button that fit the same hole in the enclosure, as some functionality diminished in the final enclosure and it became infeasible to deconstruct and reconstruct the circuit within the enclosure after it was installed. The final Cook-it product is shown below in figures 14, 15, 16, and 17.

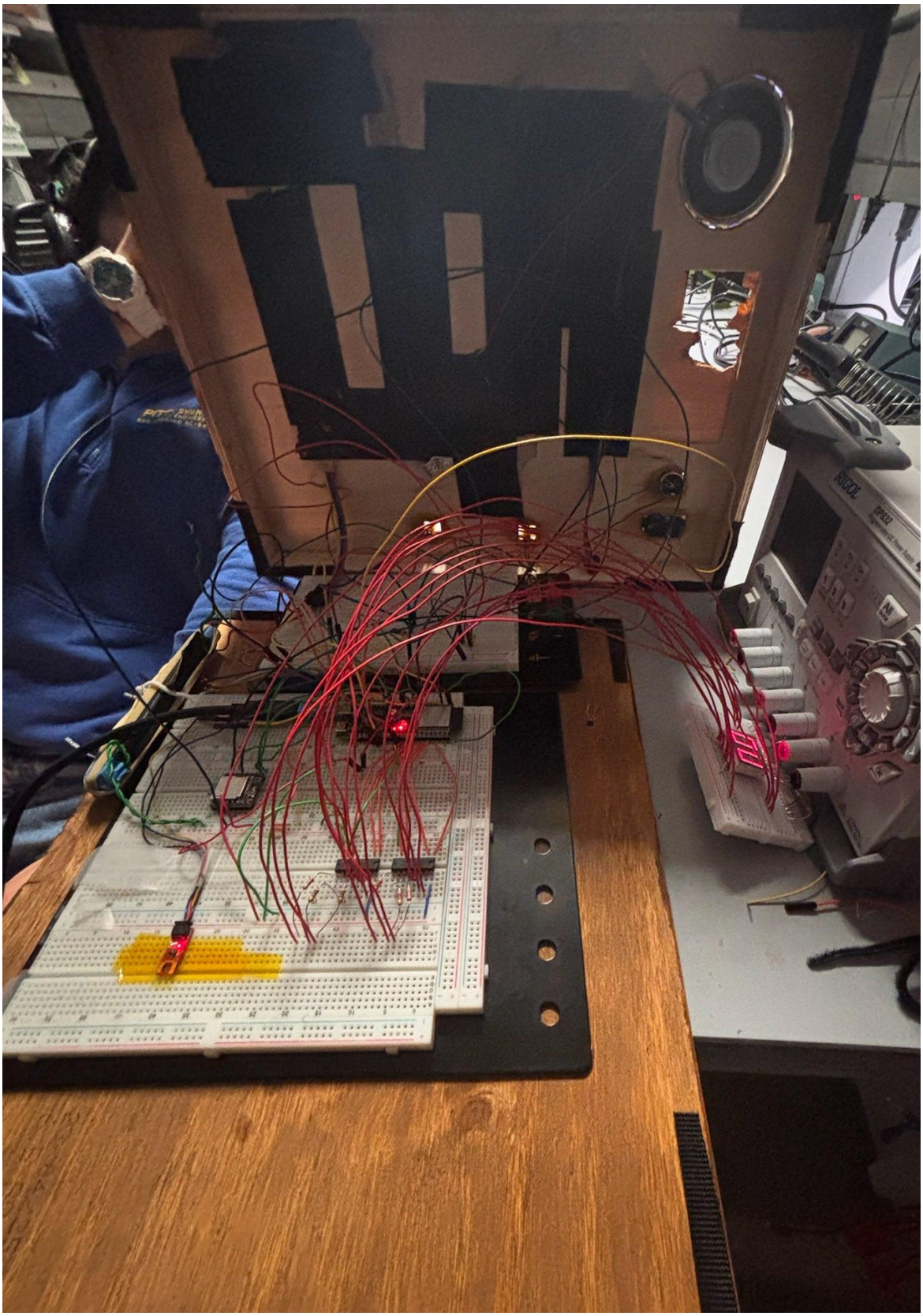

Figure 14: Cook-it Electronics

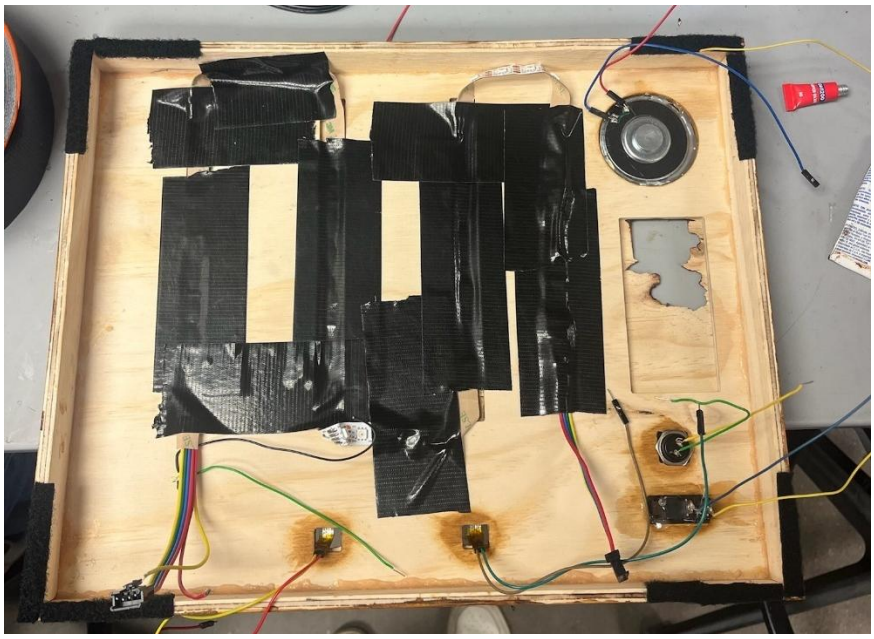

Figure 15: Cook-it Back Side

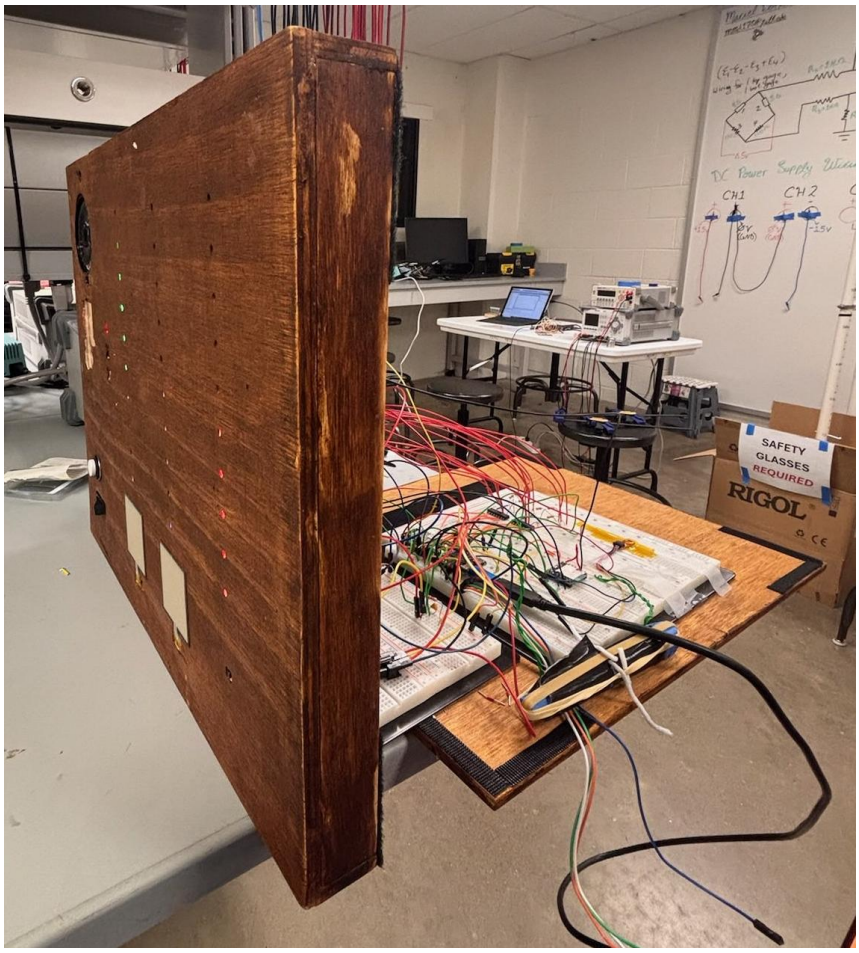

Figure 16: Cook-it Profile View

Figure 17: Cook-it Top Down View

VIII. Budget and Cost Analysis

VIII.1 – PCB BoM, Manufacturing and Assembly

The financial costs associated with the components, manufacturing and assembly of the PCB are broken down in figure 18.

Quantity	Component	Unit Price	Extended Price
10,000	PCB Unit Price	0.4	4000
1	PCB Shipping Cost	23.93	23.93
10,000	BoM	33.86	338600
60,000	PCB Assembly Cost / hour	15	900000
Total Cost			1242623.93

Figure 18: PCB Component, Manufacturing, and Assembly Cost

Each board costs $0.40 with a total shipping cost of $23.93. The shipping cost is assumed to remain constant regardless of the quantity of boards ordered. The Bill of Materials for the PCB is shown in figure 19.

Index	Quantity	Part Number	Manufacturer Part Number	Description	Customer	Available	Backorder	Unit Price	Extended Price USD
1	6	399-C0805C104J1RACTUCT-ND	C0805C104J1RACTU	CAP CER 0.1UF 100V X7R 0805	ncs53	6	0	0.41	2.46
2	1	490-10918-1-ND	GRJ21BC72A105KE11L	CAP CER 1UF 100V X7S 0805	ncs53	1	0	0.31	0.31
3	2	478-KGM21BCG2A120JTCT-ND	KGM21BCG2A120JT	CAP CER 12PF 100V C0G/NP0 0805	ncs53	2	0	0.14	0.28
4	2	445-CGA4J1X7R2A225K125ACCT-ND	CGA4J1X7R2A225K125AC	AUTOMOTIVE GRADE,MLCC,0805,100V,	ncs53	2	0	0.66	1.32
5	1	1276-6455-1-ND	CL21A106KOQNNNG	CAP CER 10UF 16V X5R 0805	ncs53	1	0	0.08	0.08
6	2	1276-6526-1-ND	CL21A226KOQNNNE	CAP CER 22UF 16V X5R 0805	ncs53	2	0	0.33	0.66
7	1	SAM12319-ND	TSW-105-22-G-S-RA	CONN HEADER R/A 5POS 2.54MM	ncs53	1	0	0.9	0.9
8	3	612-TSW-104-22-G-S-RA-ND	TSW-104-22-G-S-RA	CONN HEADER R/A 4POS 2.54MM	ncs53	3	0	0.72	2.16
9	2	612-TSW-102-22-G-S-RA-ND	TSW-102-22-G-S-RA	CONN HEADER R/A 2POS 2.54MM	ncs53	0	2	0.36	0.72
10	1	2223-MSD-4-ACT-ND	MSD-4-A	CONN MICROSD CARD R/A SMD	ncs53	1	0	0.36	0.36
11	2	445-MLF2012E100JT000CT-ND	MLF2012E100JT000	FIXED IND 10UH 15MA 800 MOHM SMD	ncs53	2	0	0.14	0.28
12	6	RMCF0805JT10K0CT-ND	RMCF0805JT10K0	RES 10K OHM 5% 1/8W 0805	ncs53	6	0	0.1	0.6
13	3	311-100KARCT-ND	RC0805JR-07100KL	RES 100K OHM 5% 1/8W 0805	ncs53	3	0	0.1	0.3
14	2	RMCF0805JT2K20CT-ND	RMCF0805JT2K20	RES 2.2K OHM 5% 1/8W 0805	ncs53	2	0	0.1	0.2
15	1	696-HVS0805-1MJ8CT-ND	HVS0805-1MJ8	RES SMD 1M OHM 5% 1/8W 0805	ncs53	1	0	2.25	2.25
16	14	YAG3707CT-ND	AC0805FR-07200RL	RES SMD 200 OHM 1% 1/8W 0805	ncs53	14	0	0.01	0.14
17	1	541-52.3KCCT-ND	CRCW080552K3FKEA	RES SMD 52.3K OHM 1% 1/4W 0805	ncs53	1	0	0.1	0.1
18	3	311-51KARCT-ND	RC0805JR-0751KL	RES 51K OHM 5% 1/8W 0805	ncs53	3	0	0.1	0.3
19	1	5407-ESP32-S3-WROOM-1-N8R8CT-ND	ESP32-S3-WROOM-1-N8R8	RF TXRX MODULE BT PCB TRACE SMD	ncs53	1	0	6.13	6.13
20	1	828-1091-1-ND	BMI270	IMU ACCEL/GYRO I2C/SPI 14LGA	ncs53	1	0	3.73	3.73
21	1	MAX98357AETE+TCT-ND	MAX98357AETE+T	IC AMP CLASS D MONO 3.2W 16TQFN	ncs53	1	0	3.19	3.19
22	2	296-3712-5-ND	SN74LS47N	IC DRVR 7 SEGMENT 16DIP	ncs53	2	0	1.87	3.74
23	2	296-39451-1-ND	TPS62175DQCR	IC REG BUCK BST ADJ 500MA 10WSON	ncs53	2	0	1.52	3.04
24	1	535-9542-1-ND	ABS07-32.768KHZ-T	CRYSTAL 32.7680KHZ 12.5PF SMD	ncs53	1	0	0.61	0.61
Total Cost									33.86

Figure 19: PCB Bill of Materials

Each PCB requires $33.86 in components. The cost associated with assembling the PCB was estimated to be $15 per hour. The PCB uses SMD components and the components are distributed between the two sides of the board. An oven was used to solder the first side of the board, and a heat gun was used to solder the second side of the board. The package sizes of the components are small and successful assembly of the board requires prior experience, resulting in an estimated labor cost of $15 per hour. The single PCB took five hours to assemble, and an additional hour was added to the total assembly time as a buffer if components need to be re-soldered.

VIII.2 – Enclosure Materials and Manufacturing

The financial costs associated with manufacturing the enclosure is broken down in figure 20.

Quantity	Component	Unit Price	Extended Price
20,000	20x20x0.125 inch birch plywood	10	200000
20,000	Enclosure Manufacturing Cost / hour	8	160000
2,000	Wood Stain	4.53	9060
Total Cost			369060

Figure 20: PCB Material and Manufacturing Cost

The enclosure is made of birch plywood and requires two 20x20x0.125-inch sheets of plywood. Each sheet costs $10. The enclosure was manufactured using an X-Carve CNC machine and the top, bottom, and sides of the enclosure were manufactured separately. The bottom piece of the enclosure is where the sensors, PCB, and LEDs were housed and consisted of advanced cutouts that took an hour and fifteen minutes to manufacture. The sides and top of the enclosure were simple cutouts that were cut out of the same plywood sheet and took a total of forty-five minutes to manufacture. The cost associated with time spent manufacturing the enclosure was estimated to be $8 per hour. The X-Carve CNC machine requires specialized training to use but after being trained to use the machine, the process of using it is rather straightforward and does not require much focus or attention. Access to an X-Carve CNC machine is assumed in this budget analysis.

VIII.3 – Total Cost of Producing 10,000 Copies

The total cost to produce 10,000 copies of the product is estimated to be $1,611,683.93. A complete breakdown of the cost is shown in figure 21.

Quantity	Component	Unit Price	Extended Price
10,000	PCB Unit Price	0.4	4000
1	PCB Shipping Cost	23.93	23.93
10,000	BoM	33.86	338600
60,000	PCB Assembly Cost / hour	15	900000
20,000	20x20x0.125 inch birch plywood	10	200000
20,000	Enclosure Manufacturing Cost / hour	8	160000
2,000	Wood Stain	4.53	9060
Total Cost			1611683.93

Figure 21: Total Cost to Produce 10,000 Units

IV. Team

IV.1 – Team Roles

Rather than assigning each team member one specific role, the team found it more productive to work collaboratively on design and manufacturing to streamline integration throughout the entire process. An exception to this was that Nicholas was solely responsible for designing, assembling, and testing the PCB. Nicholas also worked with Mari to design and manufacture the enclosure. Yasseen and Mari worked together to integrate the sensors and write the software with Mari focusing primarily on sensor integration and Yasseen on software development.

IV.2 – Team Communication

The team took advantage of in-class work time to hold informal meetings about progress made since the last meeting and next steps. Outside of class, the team communicated over text and held virtual meetings over Microsoft Teams once a week to distribute work at each stage of the project and make sure all members were on the same page. During the prototype development phase, most of the code was modularized and the GitHub repository shown in figure 22 was used as a common place to organize the project files and allow all members to stay up to date with the work being done. Once integration began, more emphasis was placed on proper version control as team members started to work on the same files and changes were made more rapidly.

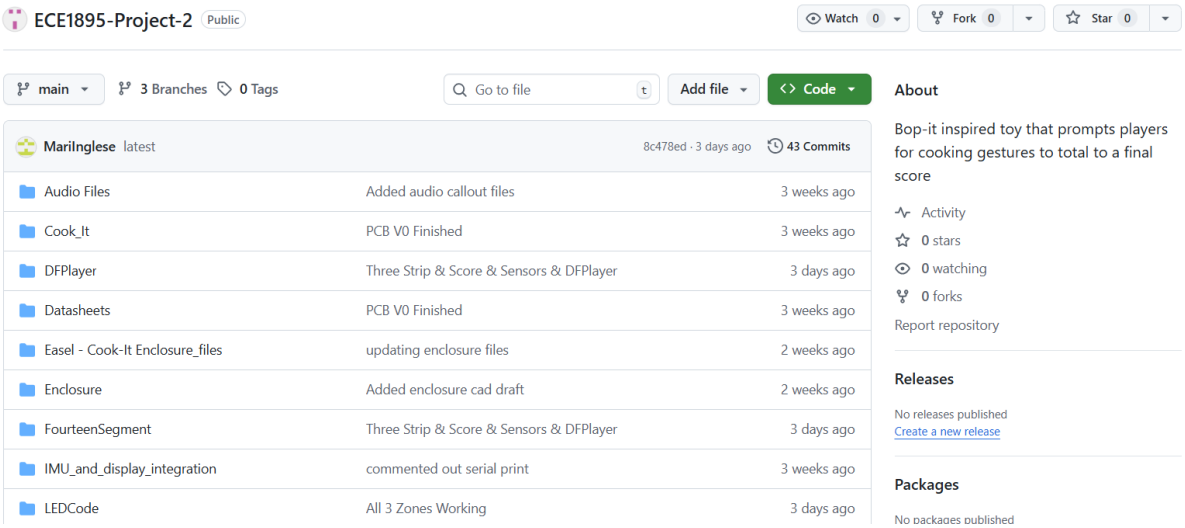

Figure 22: GitHub Repository

X. Timeline

Week 1 (Sept 28 – Oct 4): The team synthesized the original concept, brainstormed ideas, created sketches, and purchased preliminary components for the prototype.

Week 2 (Oct 5 – Oct 11): Each sensor and output was validated individually through prototyping. During this time, the team also developed a full integration plan for combining all components.

Week 3 (Oct 12 – Oct 18): Sensor validation and output work were completed. The team began designing the PCB based on this information and started integrating software from the validated component code. Enclosure design officially began, and audio circuit plans were investigated.

Week 4 (Oct 19 – Oct 25): The gameplay loop software was developed using a digital twin of the game. Individual component functions were finalized and integrated into the digital twin. The enclosure design was completed, the PCB was finished, and DFPlayer audio module experimentation began.

Weeks 5-6 (Oct 26 – Nov 8): The team received the fabricated PCB and attempted to integrate all components. The digital twin software was augmented to receive inputs from sensors and output to the LED strip, DFPlayer, and seven-segment display. Integration proved difficult, and the IMU did not always function as intended for the "Clear the Plate" minigame when mounted in the enclosure.

Week 7 (Nov 9– Nov 16): The final software and breadboard prototype were working, but the PCB remained non-functional. The team reformatted the breadboard prototype, soldered and fastened components for reliability, and mounted everything inside the enclosure. The haloflex sensor was abandoned and replaced with a button. The wooden enclosure was stained, and decals were added to label zones and improve aesthetics.

Week 8 (Nov 9– Nov 16): The final presentation/demonstration and report were finalized.

XI. Summary, Conclusions, and Future Work

Overall, the creation of the Cook-It toy proved an insightful and rewarding endeavor for the entire team. Considerable setbacks in the realization of this toy provided firsthand knowledge of the complexity and intricacy of electronic design and manufacturing. These setbacks include the eventual failure of the PCB and the unreliability of the hall effect and IMU sensors. With each of these setbacks, we were forced to think on our feet and consider a range of possible solutions or alternatives while still staying true to the initial goal. Despite the shortcomings, however, the final product did truly realize the goals of the original concept. Users were given visual and audible commands to perform chopping or crushing motions on a wooden breadboard, and they were intermittently asked to tilt the board, simulating the action of clearing items from a cutting board into a pot or saucepan. The game progressed in difficulty as time continued, and the required chopping rhythm mixed with the increasing game speed and intermittent 'Active Zone' commands combined to create a genuinely captivating and enjoyable rhythmic focus game. We effectively operated within the Arduino framework to modify the game state virtually and visualize this game state effectively while keeping the vast majority of time in the central 'Loop' available to scan and process inputs to the hit pads. The final cutting board, made from real stained wood and fastened with low-profile Velcro adhesive, is given a youthful veneer through the final addition of printed decals. This final rendition is sturdy, functional, aesthetically pleasing, and novel in its choice of sensor and game-state visualization.

In the future, this product could be further improved by possibly removing the button for the third hit zone and replacing it with another hit pad. The different hit pads could each have a unique 'hit rhythm,' defined by the variables which set the minimum and maximum time between hits. This added complexity could lean into the rhythmic part of the game and increase functionality. Furthermore, the tilting feature could be improved to increase reliability and make the gameplay experience more enjoyable. Background music, point-accruing sound effects, and LED queues to signify valid hits and combos on the hit zones could all be added to make the overall gameplay experience more immersive. This future version would inevitably also contain an updated PCB board which did not suffer from the setbacks of the initial design. Improvements to the PCB would include adding a UART chip to avoid having to bypass the development board and using power electronics that have been tested and validated in the prototyping stage. These additions would greatly increase the sleekness and simplicity of the final design and would bring the overall quality increase significantly.

Two video links have been provided, each demonstrating different game functionality:

IMU: https://www.youtube.com/watch?v=NeJY6VMcu2c

Game Over: https://www.youtube.com/watch?v=B7GHeA5Zg8c